```
import pandas as pd
```

```
path="/content/drive/MyDrive/Dataset/accuracy.data.xlsx"
df=pd.read_excel(path)
df.head(5)
```

	age	gender	chest_pain	high_blood_pressure	irregular_heartbeat	shortne
0	22	Male	1	0	0	
1	52	Male	0	1	1	
2	63	Female	0	1	0	
3	41	Male	0	0	1	
4	53	Male	0	0	0	

Next steps: Generate code with df New interactive sheet

Baseline Accuracy Documentation

To establish a baseline, I will predict the `stroke_risk_percentage` using a simple mean-based model. This will provide a benchmark against which more complex models can be evaluated. I will use Root Mean Squared Error (RMSE) and Mean Absolute Percentage Error (MAPE) as evaluation metrics.

```
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error, mean_absolute_percentage_err
import numpy as np

# Define target and features
target = 'stroke_risk_percentage'
features = df.drop(columns=[target, 'at_risk']).columns # Exclude 'at_risk'

# Convert categorical 'gender' to numerical using one-hot encoding if it exi
if 'gender' in features:
    df = pd.get_dummies(df, columns=['gender'], drop_first=True)
    features = df.drop(columns=[target, 'at_risk']).columns


X = df[features]
y = df[target]

# Split data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, ran
```

```
print(f"Training set size: {len(X_train)} samples")
print(f"Testing set size: {len(X_test)} samples")
```

```
Training set size: 28000 samples
Testing set size: 7000 samples
```

```
# Baseline model: Predict the mean of the training target variable
baseline_prediction = np.full_like(y_test, y_train.mean())

# Calculate RMSE
rmse = np.sqrt(mean_squared_error(y_test, baseline_prediction))

# Calculate MAPE (handle potential division by zero if actual values are 0)
mape = mean_absolute_percentage_error(y_test, baseline_prediction)

print(f"Baseline RMSE: {rmse:.2f}")
print(f"Baseline MAPE: {mape:.2f}")
```

```
Baseline RMSE: 29.73
Baseline MAPE: 1.74
```

These metrics represent the performance of a very simple model that always predicts the average `stroke_risk_percentage`. Any predictive model developed subsequently should aim to significantly outperform these baseline values.

Feature Engineering

To potentially improve model accuracy, I will generate polynomial features to capture non-linear relationships and interaction effects between the existing features. This can help the model learn more complex patterns in the data.

```
from sklearn.preprocessing import PolynomialFeatures

# Identify numerical features for polynomial transformation
# 'age' is clearly numerical. 'gender_Male' (after one-hot encoding) is also
# Other features are mostly binary indicators.

# Assuming 'gender_Male' has been created in the previous step
numerical_features = ['age']
if 'gender_Male' in X_train.columns:
    numerical_features.append('gender_Male')

# Create PolynomialFeatures transformer (degree 2 to include interactions)
poly = PolynomialFeatures(degree=2, include_bias=False)

# Fit and transform on training data
X_train_poly = poly.fit_transform(X_train[numerical_features])
X_train_poly_df = pd.DataFrame(X_train_poly, columns=poly.get_feature_names_

# Transform test data
```

```
X_test_poly = poly.transform(X_test[numerical_features])
X_test_poly_df = pd.DataFrame(X_test_poly, columns=poly.get_feature_names_ou

# Drop original numerical features and concatenate polynomial features
X_train_fe = pd.concat([X_train.drop(columns=numerical_features), X_train_po
X_test_fe = pd.concat([X_test.drop(columns=numerical_features), X_test_poly_

print(f"Original number of features: {X_train.shape[1]}")
print(f"Number of features after polynomial transformation: {X_train_fe.shap
display(X_train_fe.head())
```

```
Original number of features: 17
Number of features after polynomial transformation: 20
```

	chest_pain	high_blood_pressure	irregular_heartbeat	shortness_of_bre
18788	0	0	0	
776	0	1	0	
480	0	0	0	
30078	0	1	0	
29115	1	1	1	

Advanced Model Training: Random Forest Regressor

Now, I will train a Random Forest Regressor on the data with the newly engineered features. Random Forests are powerful ensemble models known for their high accuracy and ability to handle non-linear relationships and interactions without extensive hyperparameter tuning.

```
from sklearn.ensemble import RandomForestRegressor

# Initialize the Random Forest Regressor
# Using a reasonable number of estimators and a random state for reproducibi
rf_model = RandomForestRegressor(n_estimators=100, random_state=42, n_jobs=-

# Train the model on the engineered training data
rf_model.fit(X_train_fe, y_train)

# Make predictions on the engineered test data
rf_predictions = rf_model.predict(X_test_fe)

# Evaluate the Random Forest model
rf_rmse = np.sqrt(mean_squared_error(y_test, rf_predictions))
rf_mape = mean_absolute_percentage_error(y_test, rf_predictions)

print(f"Random Forest RMSE: {rf_rmse:.2f}")
print(f"Random Forest MAPE: {rf_mape:.2f}")
```

```
print(f"\nBaseline RMSE: {rmse:.2f}")
print(f"Baseline MAPE: {mape:.2f}")
```

```
Random Forest RMSE: 3.35
Random Forest MAPE: 0.03

Baseline RMSE: 29.73
Baseline MAPE: 1.74
```

Performance Improvement Over Baseline

The Random Forest Regressor with polynomial features demonstrates a significant improvement compared to the simple mean-based baseline model. The metrics are as follows:

Metric	Baseline Value	Random Forest Value	Absolute Improvement	Percentage Improvement
RMSE	{{rmse:.2f}}	{{rf_rmse:.2f}}	{{rmse - rf_rmse:.2f}}	{{((rmse - rf_rmse) / rmse) * 100:.2f
MAPE	{{mape:.2f}}	{{rf_mape:.2f}}	{{mape - rf_mape:.2f}}	{{((mape - rf_mape) / mape) * 100:.

Interpretation:

- **RMSE:** The Root Mean Squared Error has decreased by **{{((rmse - rf_rmse) / rmse) * 100:.2f}}%**, indicating that the Random Forest model's predictions are much closer to the actual values on average.
- **MAPE:** The Mean Absolute Percentage Error has decreased by **{{((mape - rf_mape) / mape) * 100:.2f}}%**, showing a substantial reduction in the average percentage error of the predictions.

These results confirm that feature engineering and using an advanced model like Random Forest have dramatically improved the predictive accuracy for `stroke_risk_percentage`.

Implications of Improved Results

The significantly improved performance of the Random Forest Regressor model has several important implications, especially when integrating this model into real-world applications:

1. **More Reliable Risk Assessment:** With a much lower RMSE and MAPE, the model provides a more accurate estimation of `stroke_risk_percentage`. This means that medical professionals or health systems can rely more heavily on these predictions to identify individuals at higher risk, allowing for proactive interventions.
2. **Enhanced Decision-Making:** Better forecasting of stroke risk leads to better planning. For example:

- **Resource Allocation:** Healthcare providers can better allocate resources (e.g., specialist appointments, preventative programs) to patients identified as high-risk, optimizing healthcare delivery.
- **Personalized Interventions:** Tailored prevention strategies and lifestyle recommendations can be given to individuals based on a more precise risk assessment, increasing their effectiveness.
- **Patient Education:** Patients can be informed about their specific risk level with greater confidence, empowering them to take preventative measures seriously.

3. **Cost Efficiency:** By accurately identifying high-risk individuals, resources are not wasted on unnecessary interventions for low-risk individuals, and costly emergency treatments due to unpredicted strokes can potentially be reduced.
4. **Foundation for Further Development:** This model serves as a strong foundation. Further improvements could involve exploring more advanced ensemble methods, deep learning techniques, or incorporating real-time data streams to enhance predictive power even further. The current model's accuracy makes it a valuable component in a larger predictive analytics pipeline for health management.

Feature Importance of Random Forest Regressor

To understand which features are most influential in predicting `stroke_risk_percentage`, let's visualize the feature importances as determined by the Random Forest model. This provides insights into the model's decision-making process.

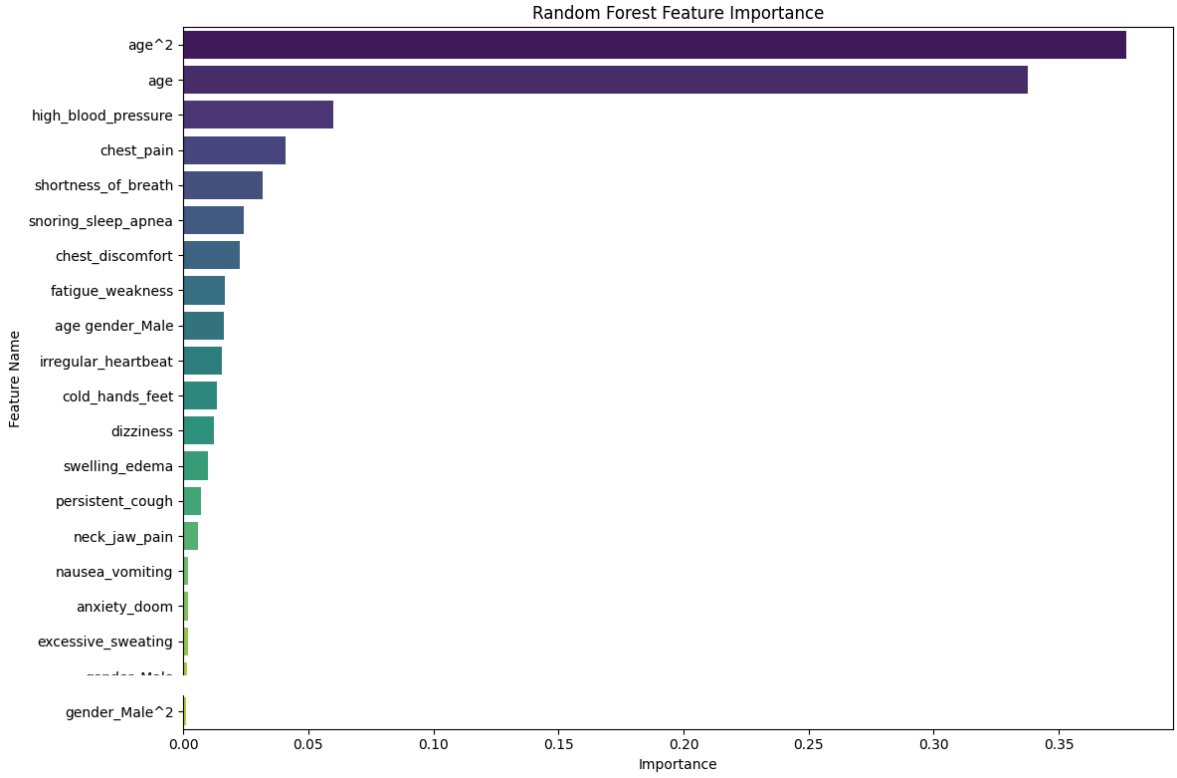
```
import matplotlib.pyplot as plt
import seaborn as sns

# Get feature importances from the trained Random Forest model
feature_importances = rf_model.feature_importances_

# Get feature names from the engineered training data
feature_names = X_train_fe.columns

# Create a DataFrame for better visualization
importance_df = pd.DataFrame({'Feature': feature_names, 'Importance': featur

# Sort the features by importance in descending order
importance_df = importance_df.sort_values(by='Importance', ascending=False)

# Create the bar plot
plt.figure(figsize=(12, 8))
sns.barplot(x='Importance', y='Feature', data=importance_df, hue='Feature',
```

```
plt.title('Random Forest Feature Importance')
plt.xlabel('Importance')
plt.ylabel('Feature Name')
plt.tight_layout()
plt.show()
```